

HTW Berlin
Fachbereich 4 – Informatik, Kommunikation und Wirtschaft
Angewandte Informatik

**Dokumentation zum Semesterprojekt in Mobile Betriebssysteme und
Netzwerke:**

Wanderwege

Nachgereicht von:
Anel Ibragim, 587828

Wanderwege.....	0
1. Beschreibung.....	2
2. GUI.....	3
3. Beschreibung.....	6
4. Komponentendiagramm.....	7
Beschreibung von Komponenten und Testverfahren.....	8
GUI (View).....	8
Beschreibung:.....	8
Testen:.....	8
MapHandler.....	8
Beschreibung:.....	8
Testverfahren:.....	8
WanderwegeController (Controller).....	9
Beschreibung:.....	9
Testverfahren:.....	9
Test von getSightseeingDetails(int id):.....	9
WanderwegeModel (Model).....	15
Beschreibung:.....	15
CommentModelTest.....	16
RatingModelTest.....	17
Test: testGetAllWanderwegsMarks.....	18
WanderwegeSpeicher (Database).....	19
Testverfahren:.....	19
Es wird davon ausgegangen, dass die androidx.room-Datenbank stabil funktioniert und die getter, setter-Methoden in den Entitäten-Klassen, auch Queries und Insert-Methoden in Interfaces und abstrakte AppDataBase-Klasse von den Herstellern schon getestet wurden.....	19
5. Schnittstellen.....	19
Interfaces von WanderwegeController.....	19
Interface IWanderwegController.....	19
Interface IComRatController.....	21
Interfaces von WanderwegeModel.....	21
Interface ICommentModel.....	21
Interface IRatingModel.....	21
Interface IMarkOfWanderwegModel.....	22
Interface ISightseeingModel.....	22
Interface IWadnerwegModel.....	23
Interfaces von MapHandler.....	24
Interfaces von WanderwegeSpeicher.....	24
CommentDao.....	24
MarkOfSightseeingDao.....	24
MarkOfWanderwegDao.....	25
RatingDao.....	25
SightseeingDao.....	25
WanderwegDao.....	25

6. End-To-End-Tests.....	26
Use Case 1:.....	26
Use Case 2:.....	27
Use Case 3:.....	27
Use Case 4:.....	28

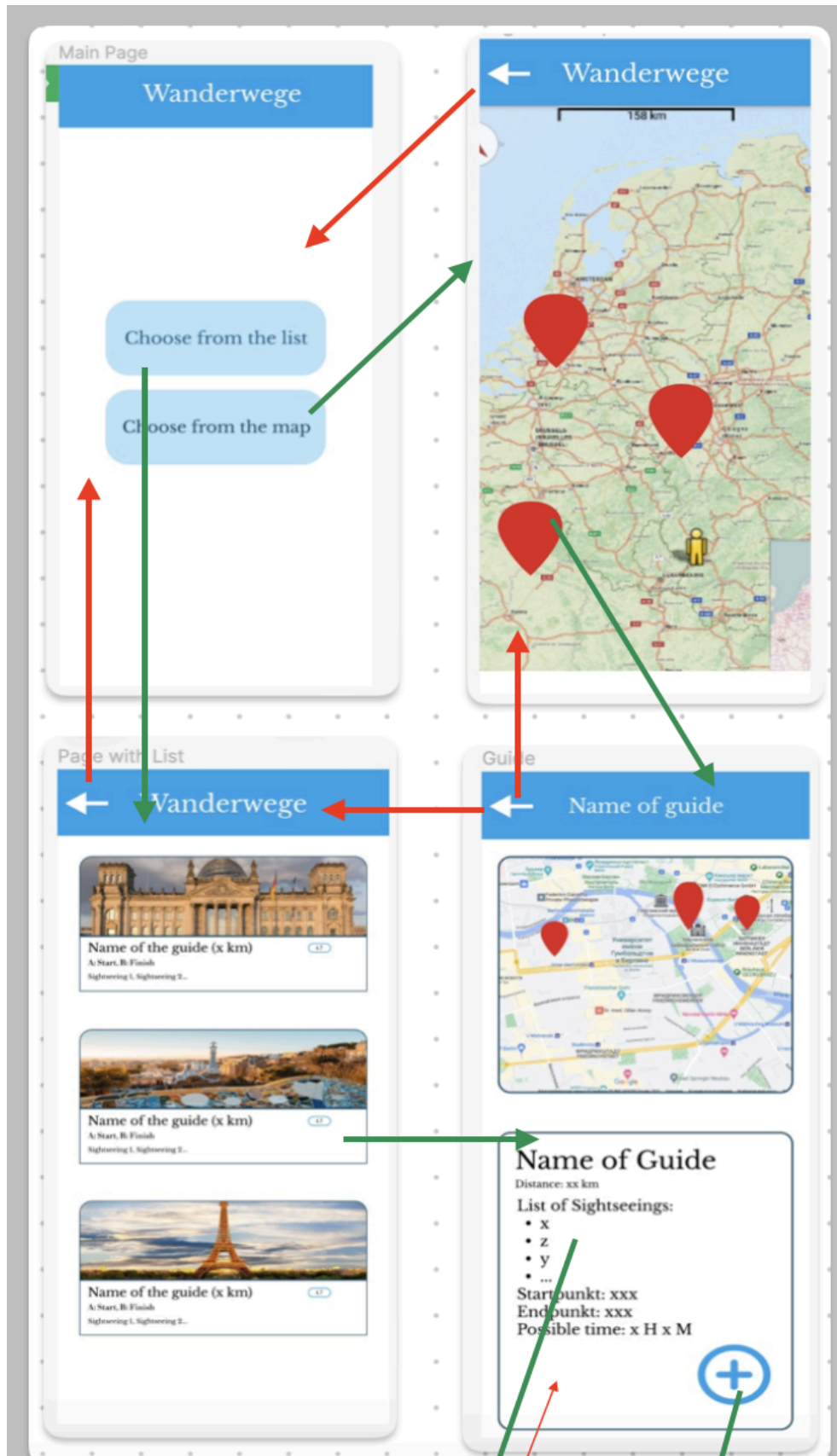
1. Beschreibung

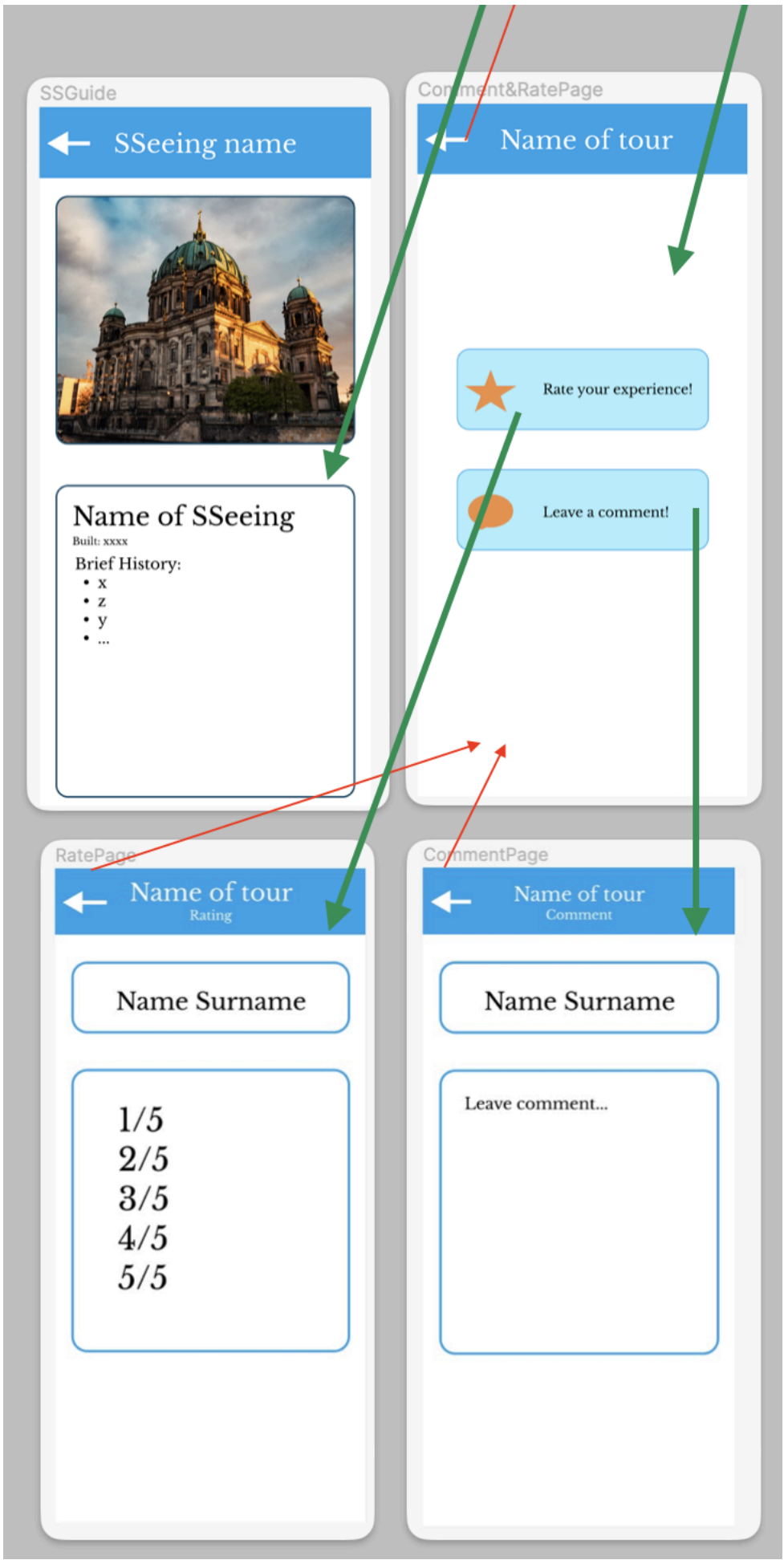
In dieser App kann man sich Wanderwege in verschiedenen Ländern und Städten anschauen und auswählen. Zu jedem Wanderweg kann man Informationen über Sehenswürdigkeiten und Wegbeschreibungen wie beispielsweise das Baujahr, eine kurze Beschreibung, den Start- und Endpunkt des Weges, die Distanz sowie die benötigte Zeit nachlesen. Außerdem kann man diese Sehenswürdigkeiten bewerten und Kommentare hinzufügen.

Die Auswahl von Wanderwegen erfolgt entweder über die Karte oder aus einer Liste von Wanderwegen, die vom Ersteller zuvor in die Datenbank hinzugefügt wurden.

Man kann auch seine Position auf der Karte anzeigen lassen, und die Wanderwege in den Städten sowie die Sehenswürdigkeiten entlang des Weges sind auf der Karte mit roten Punkten markiert.

2. GUI





3. Beschreibung

Der Homescreen besteht aus dem Button „Aus Liste wählen“ und „Auf Karte auswählen“. Der erste Button zeigt eine Liste aller gespeicherten Wanderwege an, durch die man scrollen kann. Der zweite Button öffnet eine Karte, auf der bereits rote Markierungen für die Städte vorhanden sind, in denen Wanderwege verfügbar sind.

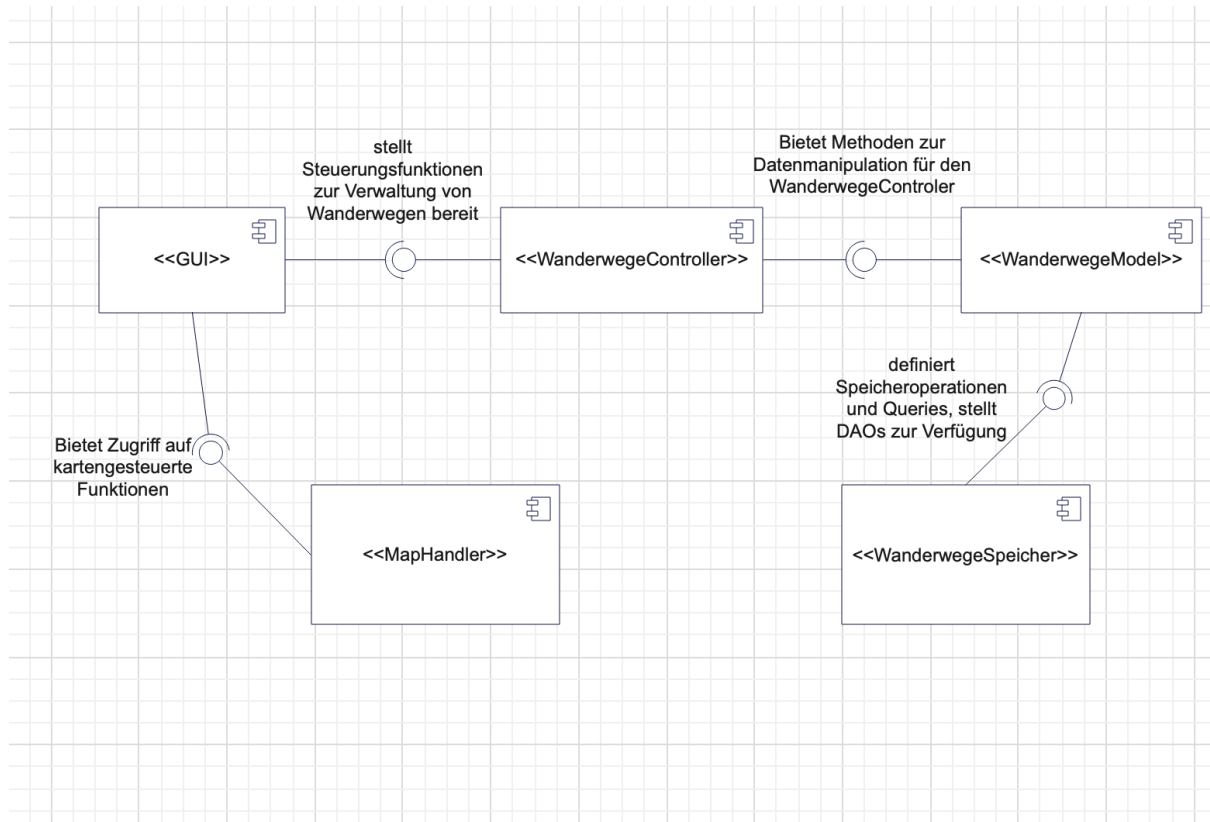
In der Liste gibt es mindestens eine sogenannte „Card“, die ein Foto vom Startpunkt sowie eine kurze Information mit Start- und Endpunkten, Distanz und dem Namen des Wanderwegs zeigt. In einer Ecke wird außerdem die durchschnittliche Bewertung angezeigt.

Wenn man auf eine der roten Markierungen auf der Karte klickt, gelangt man direkt zu den Informationen über den ausgewählten Weg.

Man kann sich jede einzelne „Card“ anschauen, dies erfolgt in einem neuen View. Dort werden eine kleine Karte mit Sehenswürdigkeiten und anderen Informationen angezeigt. Durch Betätigen des „+“-Buttons gelangt man zu einem weiteren View, in dem man selbst wählen kann, ob man einen Kommentar und/oder eine Bewertung hinterlassen möchte.

Wenn man eine Sehenswürdigkeit bewerten möchte, muss man seinen Namen und Vornamen eingeben, oder irgendeinen Nickname, und auch auf Rating-Button tippen. Wird kein Namen angegeben, ist die Bewertung unmöglich. Wenn man einen Kommentar schreiben möchte, muss man seinen Namen und Vornamen eingeben, oder irgendeinen Nickname, und auch einen Kommentar. Ohne Namen wird kein Kommentar gespeichert.

4. Komponentendiagramm



Beschreibung von Komponenten und Testverfahren

GUI (View)

Beschreibung:

Die GUI-Komponente ist für die Darstellung der Benutzeroberfläche verantwortlich und reagiert auf die Berührungen bzw. Eingaben des Nutzers auf dem Touchscreen. Sie zeigt dem Benutzer die Wanderwege, Sehenswürdigkeiten an und ermöglicht ihm die Interaktion mit der App. Die GUI-Komponente enthält verschiedene Views, z. B. den Homescreen, die Wanderweg-Liste, die Kartenansicht usw. Sie kommuniziert mit dem Model über dessen Interface, um Benutzeraktionen entgegenzunehmen und die aktuellen Daten anzuzeigen.

Testen:

Automatisierte UI-Tests – mittels Espresso:

Mittels Espresso werden automatisierte Tests durchgeführt. Diese führen Aktionen auf der Benutzeroberfläche automatisch aus und überprüfen, ob die erwarteten Ergebnisse erzielt werden. Es ist wichtig zu testen, ob korrekte Informationen angezeigt sind und dass alle Elemente auf den korrekten Stellen stehen.

MapHandler

Beschreibung:

Die MapHandler-Komponente ist für die Steuerung und Interaktion mit der Kartenansicht über die OSMDroid-Bibliothek in der Wanderwege-App verantwortlich. Sie bietet eine Schnittstelle für die GUI (View), um die Kartenansicht zu initialisieren, Marker hinzuzufügen, die Kartenposition zu ändern, die Karte zu zentrieren und Berechtigungen anzufordern.

Testverfahren:

Es wird davon ausgegangen, dass die OSMDroid-Bibliothek stabil funktioniert und von den Herstellern schon getestet wurde.

- Die Methoden `initializeMap()`, `setMapCenter()`, `addWanderwegMarkerToMap(List<MarkOfWanderweg> locations, double latitude, double longitude, int zoomLevel)`, `addSightseeingMarkerToMap(MarkOfSightseeing mark, int sightseeingId)` benutzen nur die Methoden aus der OSMDroid-Bibliothek. Daher werden sie nicht getestet.
- Die Methode `requestPermissionsIfNecessary(String[] permissions)` wurde auch aus der Bibliothek übernommen.

WanderwegeController (Controller)

Beschreibung:

Die WanderwegeController-Komponente ist ein Bindeglied zwischen dem WanderwegeModel und GUI. Sie ist verantwortlich für die Entgegennahme von Eingaben des Nutzers, deren Verarbeitung und die Weiterleitung der notwendigen Befehle an das Model oder die View, um eine entsprechende Reaktion in der App hervorzurufen. Diese Komponente bietet 2 Interfaces für die GUI, um auf Eingaben und Steuerungen des Nutzers zu reagieren und korrekte Daten anzuzeigen.

Testverfahren:

Die Teststrategien der `WanderwegeController`-Komponente in der `ControllerTests`-Package zielen darauf ab, die Funktionalität des `WanderwegeController` zu überprüfen und sicherzustellen, dass er korrekt mit den Modellen interagiert, um Daten abzurufen und anzuzeigen. Für den meisten Testfällen werden `AppDataBase` und verschiedene Controller-Klassen gemockt und mit Unit-Tests getestet, um zu überprüfen, wie die zwei Komponente miteinander agieren. Hier sind die genauere Strategien für die Tests von verschiedenen Methoden:

Test von `getSightseeingNameById(int id)`:

- **Ziel:** Überprüfen, ob der Controller korrekt den Namen einer Sehenswürdigkeit anhand ihrer ID abrufen.
- **Setup:**
 - Mocking des `AppDataBase`-Objekts und des `ISightseeingModel`.
 - Erstellung eines `WanderwegController` mit den gemockten Objekten.
- **Testablauf:**
 - Aufruf der Methode `getSightseeingNameById` des Controllers mit einer spezifischen ID.
 - Verifizierung, dass die Methode `getSightseeingNameById` des `ISightseeingModel` aufgerufen wird und den erwarteten Namen zurückgibt.

Test von `getSightseeingDetails(int id)`:

- **Ziel:** Überprüfen, ob der Controller korrekt Details zu einer Sehenswürdigkeit anhand ihrer ID abrufen.
- **Setup:**
 - Mocking des `IWanderwegModel` und `ISightseeingModel`.

- Erstellung eines `WanderwegController` mit den gemockten Modellen.
- **Testablauf:**
 - Aufruf der Methode `getSightseeingDetails` des Controllers mit einer spezifischen ID.
 - Verifizierung, dass die Methode `getSightseeingById` des `ISightseeingModel` aufgerufen wird und das erwartete `Sightseeing`-Objekt zurückgibt.

Tests von weiteren Methoden.

- Die Methoden `loadWanderwegsData()`, `loadMarkers()`, `getWanderwegDetails()`, `onItemClick()`, `getSightseeingNames()`, `getsightseeingIdsByWanderweg()`, `getMarkOfSightseeingById()`, `getWanderwegLis()` wurden nicht erfolgreich getestet, weil einer des Konstruktors des `WanderwegModels` intern `AppDataBase.getInstance(context)` aufruft und ein Android-Kontextobjekt ungleich Null erwartet.
- Während der Testphase wurde es herausgefunden, dass in der Testumgebung, insbesondere bei Verwendung von Mockito, handelt es sich jedoch bei dem bereitgestellten Kontext nicht um einen echten Android-Kontext, was zu dem `NullPointerException` führt.
- Die Frage war dann auch: warum bei anderen ausgeführten Tests, wo `AppDataBase` auch gemockt wurde, keine Exception geworfen wurde?
 - Es wurde dann klar, dass bei ausgeführten Tests wurde direkt die `AppDataBase` gemockt und die DAOs, dann wurde die simulierte Datenbank an das `WanderwegModel` übergeben. Dieser Ansatz basiert nicht auf der `AppDataBase.getInstance(context)`-Methode und stößt daher nicht auf die `NullPointerException`. Im fehlgeschlagenen Tests instanziiert der `WanderwegController`-Konstruktor intern ein `WanderwegModel`, das wiederum `AppDataBase.getInstance(context)` aufruft. Da der Kontext in Tests kein echter Android-Kontext ist, führt dies zu einer `NullPointerException`.
- Wie könnte dieses Problem gelöst werden? (wahrscheinlich, aber nicht sicher)
 - Man kann ein Test-Framework verwenden, das Android-Kontext unterstützt: eine Option ist Robolectric, das die Ausführung von

Komponententests mit einer simulierten Android-Umgebung ermöglicht und einen Scheinkontext bereitstellt. Auf diese Weise würden Aufrufe von `AppDataBase.getInstance(context)` einen Kontext ungleich Null erhalten.

- Statische Mock-Methoden (erweitert): Wenn man Mockito 3.4.0 oder neuer verwendet, kann man `MockStatic` verwenden, um den statischen `AppDataBase.getInstance(context)`-Aufruf zu verspotten. Dieser Ansatz würde das Hinzufügen einer zusätzlichen Abhängigkeit zum Mocking statischer Methoden erfordern.
- Warum wurde es nicht gemacht?

Aus zeitlichen Gründen war es nicht möglich eine neue Technologie wie Robolectric von Anfang an erlernen. Außerdem sind diese Methoden in dieser Klasse mithilfe von `WanderwegeModel`-Methoden aufgebaut, die dann aber sorgfältig getestet wurden. Es wird davon ausgegangen, dass die oben genannten nicht getestete Methoden in `WanderwegController`-Komponente trotzdem korrekt funktionieren, weil sie sich auf `WanderwegeModel`-Methoden basieren und nur aus eine bis zwei Zeilen bestehen.

Test von `submitRating(String name, int ratingValue, int wanderwegId, Runnable onSuccess, Runnable onFailure)` -

Success Case:

- **Ziel:** Überprüft das erfolgreiche Verhalten der `submitRating`-Methode bei gültigen Eingaben.
- **Setup:**
 - Ein gültiger Name, eine Bewertung und eine Wanderweg-ID werden als Eingabeparameter festgelegt.
 - `onSuccess` und `onFailure` sind Mock-Objekte für die Callbacks.
 - Verwendung eines `ArgumentCaptor`, um das `onSuccess`-Callback zu erfassen.
- **Testablauf:**

- Die `submitRating`-Methode des Controllers wird mit den definierten Parametern aufgerufen.
- Es wird verifiziert, dass die `insertRating`-Methode des `IRatingModel` genau einmal mit den entsprechenden Parametern aufgerufen wird.
- Der erfasste `onSuccess`-Callback wird manuell aufgerufen, um zu simulieren, dass die Operation erfolgreich war.
- Es wird überprüft, ob die Methode `calculateAndUpdateAverageRating` des `IRatingModel` aufgerufen wird, um die durchschnittliche Bewertung zu aktualisieren.

Test von `submitRating(String name, int ratingValue, int wanderwegId, Runnable onSuccess, Runnable onFailure)` - Failure Case:

- **Ziel:** Überprüft das Verhalten der `submitRating`-Methode, wenn ungültige Eingaben (leerer Name und/oder ungültige Wanderweg-ID) bereitgestellt werden.
- **Setup:**
 - Ungültige Eingabewerte für den Namen und die Wanderweg-ID werden festgelegt.
 - `onSuccess` und `onFailure` sind Mock-Objekte für die Callbacks.
- **Testablauf:**
 - Die `submitRating`-Methode wird mit den ungültigen Werten aufgerufen.
 - Es wird verifiziert, dass `insertRating` nicht aufgerufen wird, da die Eingabedaten ungültig sind.
 - Es wird sichergestellt, dass der `onFailure`-Callback einmal ausgeführt wird, während der `onSuccess`-Callback nie **aufgerufen wird.**

Test `loadGeoPointsForSightseeings()`:

- **Ziel:** Überprüft, ob der `MainActivityController` korrekt die Methode zum Hinzufügen von Geopunkten für Sehenswürdigkeiten aufruft.
- **Setup:**
 - Mocking des `AppDataBase`-Objekts und des `ISightseeingModel`.
 - Erstellung eines `MainActivityController` mit den gemockten Objekten.
- **Testablauf:**
 - Aufruf der Methode `loadGeoPointsForSightseeings` des `Controllers`.
 - Verifizierung, dass die Methode `addGeoPointsForSightseeings` des `ISightseeingModel` einmal aufgerufen wird.

Test `loadGeoPointsForWanderwegs`:

- **Ziel:** Überprüft, ob der `MainActivityController` korrekt die Methode zum Hinzufügen von Geopunkten für Wanderwege aufruft.
- **Setup:**
 - Mocking des `WanderwegModel`.
 - Erstellung eines `MainActivityController` mit dem gemockten Modell.
- **Testablauf:**
 - Aufruf der Methode `loadGeoPointsForWanderwegs` des `Controllers`.
 - Verifizierung, dass die Methode `addGeoPointsForWanderwegs` des `WanderwegModel` einmal aufgerufen wird.

Test onDestroy():

- **Ziel:** Überprüft, ob der `MainActivityController` bei der Zerstörung den `ExecutorService` ordnungsgemäß herunterfährt.
- **Setup:**
 - Mocking des `ExecutorService`.
- Erstellung eines `MainActivityController` mit dem gemockten `ExecutorService`.
- **Testablauf:**
 - Aufruf der `onDestroy`-Methode des Controllers.
 - Verifizierung, dass die `shutdown`-Methode des `ExecutorService` einmal aufgerufen wird.

Test submitComment(String name, String commentText, Runnable onSuccess, Runnable onFailure) - Success Case:

- **Ziel:** Überprüft das erfolgreiche Verhalten der `submitComment`-Methode, wenn gültige Eingabewerte bereitgestellt werden.
- **Setup:**
 - Verwendung von Mocks für das `ICommentModel` sowie für die `onSuccess`- und `onFailure`-Callbacks.
 - Initialisierung des `CommentActivityController` mit dem gemockten Modell.
- **Testablauf:**
 - Aufruf der `submitComment`-Methode mit einem gültigen Namen und Kommentartext.
 - Überprüfung, ob das `insertComment` des `ICommentModel` genau einmal mit einem `Comment`-Objekt und den korrekten Callbacks aufgerufen wird.
 - Sicherstellung, dass weder `onSuccess` noch `onFailure` ausgeführt werden (da diese erst nach dem

erfolgreichen/fehlgeschlagenen Einfügen des Kommentars aufgerufen werden sollten).

Test submitComment - Failure Case:

- **Ziel:** Überprüft das Verhalten der `submitComment`-Methode bei ungültigen Eingaben (leerer Name und/oder leerer Kommentartext).
- **Setup:**
 - Gleiche Mock-Konfiguration wie im vorherigen Test.
- **Testablauf:**
 - Aufruf der `submitComment`-Methode mit leeren Werten für den Namen und den Kommentartext.
 - Überprüfung, ob das `insertComment` des `ICommentModel` niemals aufgerufen wird, da die Eingabedaten ungültig sind.
 - Sicherstellung, dass der `onFailure`-Callback einmal ausgeführt wird und `onSuccess` nie aufgerufen wird.

WanderwegeModel (Model)

Beschreibung:

Die `WanderwegeModel`-Komponente ist für die Logik der Anwendung verantwortlich. Diese Logik gibt an, wie die Daten geschaffen, gespeichert und aus dem Datenbank rausgeholt werden dürfen. Sie speichert und ruft Wanderweginformationen, Sehenswürdigkeiten, Kommentare, Bewertungen und relevante Daten (wie z.B. Start- und Endpunkte, Distanz, Zeit, durchschnittliche Bewertung, IDs) für diese Entitäten ab.

Testverfahren:

Die Teststrategien der `WanderwegeModel`-Komponente in der `ModelTests`-Package zielen darauf ab, die Funktionalität des `WanderwegeModel` zu überprüfen und sicherzustellen, dass er korrekt mit der Datenbank interagiert, um Daten aus dem Datenbank abzurufen. Für den meisten Testfällen werden `AppDataBase` und verschiedene `CommentDao`-Interfaces gemockt und mit Unit-Tests getestet, um zu überprüfen, wie die zwei Komponente(Model und Speichehr) miteinander agieren. Hier sind die genauere Strategien für die Tests von verschiedenen Methoden:

CommentModelTest

- **Test insertComment(Comment comment, Runnable onSuccess, Runnable onFailure) - Success Case:**
 - **Ziel:** Überprüft, ob die insertComment-Methode erfolgreich einen Kommentar in die Datenbank einfügt.
 - **Setup:**
 - Mocking von AppDataBase und CommentDao.
 - Erstellung eines CommentModel mit dem gemockten Datenbankobjekt.
 - **Testablauf:**
 - Einrichtung des Mocks, um keine Aktionen beim Einfügen eines Kommentars auszuführen (erfolgreiche Simulation).
 - Aufruf der insertComment-Methode mit einem Kommentarobjekt und Mock-Callbacks.
 - Verwendung eines CountDownLatch, um die asynchrone Natur der Operation zu berücksichtigen.
 - Überprüfung, dass der onSuccess-Callback ausgeführt wird und der onFailure-Callback nicht.
- **Test insertComment(Comment comment, Runnable onSuccess, Runnable onFailure) - Failure Case:**
 - **Ziel:** Überprüft das Verhalten der insertComment-Methode bei einem Fehler während des Einfügens.
 - **Testablauf:**
 - Einrichtung des Mocks, um eine RuntimeException beim Einfügen eines Kommentars zu werfen.
 - Ähnliche Schritte wie im vorherigen Test, aber mit dem Erwartungswert, dass der onFailure-Callback ausgeführt wird und onSuccess nicht.

RatingModelTest

- **Test insertRating(Rating rating, Runnable onSuccess, Runnable onFailure) - Success Case:**
 - **Ziel:** Überprüft, ob insertRating erfolgreich ein Rating in die Datenbank einfügt.
 - **Testablauf:**
 - Aufruf von insertRating mit einem Mock-Rating und Mock-Callbacks.
 - Überprüfung, ob die insertRating-Methode des RatingDao einmal aufgerufen wird.

- Überprüfung, dass der `onSuccess`-Callback ausgeführt wird und `onFailure` nicht.
- **Test `insertRating(Rating rating, Runnable onSuccess, Runnable onFailure)` - Failure Case:**
 - **Ziel:** Überprüft das Verhalten von `insertRating` bei einem Fehler.
 - **Testablauf:**
 - Einrichtung des Mocks, um eine `RuntimeException` beim Einfügen zu werfen.
 - Überprüfung, dass der `onFailure`-Callback ausgeführt wird und `onSuccess` nicht.
- **Test `calculateAndUpdateAverageRating(int wanderwegId)`:**
 - **Ziel:** Überprüft, ob die Methode `calculateAndUpdateAverageRating` korrekt das Durchschnittsrating berechnet und in der Datenbank aktualisiert.
 - **Setup:**
 - Mocking von `RatingDao` und Vorbereitung von Rating-Daten.
 - **Testablauf:**
 - Aufruf der `calculateAndUpdateAverageRating`-Methode mit einer Wanderweg-ID.
 - Einrichtung des Mocks, um eine Liste von Bewertungen zurückzugeben, wenn nach Bewertungen für einen bestimmten Wanderweg gefragt wird.
 - Berechnung des erwarteten Durchschnittswertes basierend auf den gemockten Daten.
 - Überprüfung, ob die `updateAverageRating`-Methode des `RatingDao` mit dem berechneten Durchschnittswert aufgerufen wird.

MarkOfWanderwegModelTest

- **Test `getAllWanderwegsMarks()`**
 - **Ziel des Tests:** Überprüfung, ob die Methode `getAllWanderwegsMarks` des `MarkOfWanderwegModel` korrekt eine Liste von `MarkOfWanderweg`-Objekten aus der Datenbank abrufen.
 - Testsetup
 - **Mocking:**

- `AppDataBase`: Die Datenbank wird gemockt, um die Abhängigkeit von einer echten Datenbank zu vermeiden.
 - `MarkOfWanderwegDao`: Das DAO (Data Access Object) für `MarkOfWanderweg` wird gemockt, um die Datenbankinteraktionen zu simulieren.
- **Modell-Initialisierung:**
 - Eine Instanz von `MarkOfWanderwegModel` wird erstellt und mit dem gemockten `AppDataBase`-Objekt initialisiert.
- **Testablauf**
 - **Vorbereitung der Testdaten:**
 - Eine Liste von `MarkOfWanderweg`-Objekten (`expectedMarks`) wird erstellt, um die erwarteten Rückgabewerte zu simulieren.
 - `when(mockDao.getAllWanderwegsMarks()).thenReturn(expectedMarks)`: Hier wird das Verhalten des gemockten `MarkOfWanderwegDao` so eingestellt, dass es die vorbereitete Liste von Markierungen zurückgibt, wenn die Methode `getAllWanderwegsMarks` aufgerufen wird.
 - **Ausführung der Methode unter Test:**
 - Die Methode `getAllWanderwegsMarks` des Modells `MarkOfWanderwegModel` wird aufgerufen, und das Ergebnis wird in `actualMarks` gespeichert.
 - **Überprüfung des Ergebnisses:**
 - `assertEquals(expectedMarks, actualMarks)`: Der Test überprüft, ob die tatsächlich zurückgegebenen Markierungen (`actualMarks`) mit den erwarteten Markierungen (`expectedMarks`) übereinstimmen.

WanderwegModelTest

- **Test von `getAllWanderwegs()`:**
 - **Ziel:** Überprüfen, ob das Modell korrekt alle Wanderwege aus der Datenbank abrufen.
 - **Setup:** Mocking des `AppDataBase` und der DAOs (`WanderwegDao`, `SightseeingDao`, `MarkOfWanderwegDao`).
 - **Testablauf:**

- Aufruf der Methode `getAllWanderwegs` des Modells.
- Verifizierung, dass die Methode `getAllWanderwegs` des `WanderwegDao` aufgerufen wird und eine Liste von Wanderwegen zurückgibt, die den erwarteten Ergebnissen entspricht.
- **Test von `getWanderwegById(int id)`:**
 - **Ziel:** Überprüfen, ob das Modell einen spezifischen Wanderweg anhand seiner ID korrekt abrufen.
 - **Setup:** Ähnlich wie beim vorherigen Test, Mocking der notwendigen Komponenten.
 - **Testablauf:**
 - Aufruf der Methode `getWanderwegById` mit einer spezifischen Wanderweg-ID.
 - Verifizierung, dass die Methode `getWanderwegById` des `WanderwegDao` aufgerufen wird und ein `Wanderweg`-Objekt zurückgibt, das dem erwarteten Ergebnis entspricht.
- **Test von `getSightseeingIdsByWanderwegId(int wanderwegId)`:**
 - **Ziel:** Überprüfen, ob das Modell korrekt die IDs der Sehenswürdigkeiten für einen spezifischen Wanderweg abrufen.
 - **Setup:** Mocking der Datenbank und der DAOs.
 - **Testablauf:**
 - Aufruf der Methode `getSightseeingIdsByWanderwegId` mit einer spezifischen Wanderweg-ID.
 - Verifizierung, dass die Methode `getSightseeingIdsByWanderwegId` des `WanderwegDao` aufgerufen wird und eine Liste von IDs zurückgibt, die den erwarteten Ergebnissen entspricht.
- **Test von `addGeoPointsForWanderwegs()`:**
 - **Ziel:** Überprüfen, ob das Modell korrekt GeoPoints für Wanderwege hinzufügt.
 - **Setup:** Mocking des `ExecutorService` und `MockOfWanderwegDao`.
 - **Testablauf:**

- Ausführung der Methode `addGeoPointsForWanderwegs`.
- Verwendung von `CountDownLatch` um auf die Vollendung der asynchronen Aufgabe zu warten.
- Verifizierung, dass die Methode `insert` des `MarkOfWanderwegDao` die korrekte Anzahl an Malen mit den richtigen Parametern aufgerufen wird.

SightseeingModelTest:

Test von `getSightseeingById(int id)`:

- **Ziel:** Überprüfen, ob das Modell korrekt ein Sightseeing-Objekt anhand seiner ID aus der Datenbank abrufen.
- **Setup:** Mocking des `AppDataBase` und des `SightseeingDao`.
- **Testablauf:**
 - Aufruf der Methode `getSightseeingById` des Modells mit einer spezifischen Sightseeing-ID.
 - Verifizierung, dass die Methode `getSightseeingById` des `SightseeingDao` aufgerufen wird und das erwartete Sightseeing-Objekt zurückgibt.

Test von `getSightseeingNameById(int id)`:

- **Ziel:** Überprüfen, ob das Modell korrekt den Namen eines Sightseeing-Objekts anhand seiner ID abrufen.
- **Setup:** Mocking des `AppDataBase` und des `SightseeingDao`.
- **Testablauf:**
 - Aufruf der Methode `getSightseeingNameById` mit einer spezifischen Sightseeing-ID.
 - Verifizierung, dass die Methode `getSightseeingNameById` des `SightseeingDao` aufgerufen wird und den erwarteten Namen zurückgibt.

Test von `getSightseeingNames(List<Integer>)`:

- **Ziel:** Überprüfen, ob das Modell korrekt die Namen mehrerer Sightseeing-Objekte anhand ihrer IDs abrufen.
- **Setup:** Mocking des `AppDataBase` und des `SightseeingDao`.
- **Testablauf:**
 - Aufruf der Methode `getSightseeingNames` mit einer Liste von IDs.
 - Verifizierung, dass die Methode `getSightseeingNameById` des `SightseeingDao` für

jede ID in der Liste aufgerufen wird und eine Liste der erwarteten Namen zurückgibt.

Test von `getMarkOfSightseeingById(int id)`:

- **Ziel:** Überprüfen, ob das Modell korrekt die Markierungsdaten eines bestimmten Sightseeing-Objekts anhand seiner ID abrufen.
- **Setup:** Mocking des `AppDataBase` und des `MarkOfSightseeingDao`.
- **Testablauf:**
 - Aufruf der Methode `getMarkOfSightseeingById` mit einer spezifischen Mark-ID.
 - Verifizierung, dass die Methode `getMarkOfSightseeingById` des `MarkOfSightseeingDao` aufgerufen wird und die erwarteten Markierungsdaten zurückgibt.

WanderwegeSpeicher (Database)

Beschreibung:

Die WanderwegeSpeicher-Komponente ist nur für die Datenbank verantwortlich. Diese Komponente enthält Klassen für Entitäten wie `Wanderweg`, `Sightseeing`, `MarkOfWanderweg`, `MarkOfSightseeing`, `Rating`, `Comment`. Es wird für die Datenbank `androidx.room` verwendet. Diese Komponente bietet getter- und setter-Methoden zum Aufrufen und Speichern von Daten in der Datenbank. Diese Komponente bietet eine klar definierte Schnittstelle für den Zugriff auf die in der Datenbank gespeicherten Daten durch die DAO-Klassen.

Testverfahren:

Es wird davon ausgegangen, dass die `androidx.room`-Datenbank stabil funktioniert und die getter, setter-Methoden in den Entitäten-Klassen, auch Queries und Insert-Methoden in Interfaces und abstrakte `AppDataBase`-Klasse von den Herstellern schon getestet wurden.

5. Schnittstellen

Interfaces von `WanderwegeController`

`WanderwegController` dient als zentrale Stelle zwischen der Benutzeroberfläche und den Modellen einer Anwendung. Diese Komponente stellt 2 Interfaces bereit, damit diese von

GUI-Komponente benutzt werden können. Mithilfe diesen Interfaces werden die Daten vom Model geholt, verarbeitet nach Bedarf und der View zur Verfügung gestellt. Es wurde 2 Interfaces definiert, damit der Code leicht zu verstehen und zu warten ist und damit jedes Interface und dann die implementierten Klassen sich nur um eine Sache kümmern.

Interface IWanderwegController:

Dieses Interface befasst sich mit der Verwaltung von Wanderwegen und Sightseeings, damit sie korrekterweise angezeigt werden können.

1. **void loadWanderwegsData () :**
 - Ruft eine Liste aller Wanderweg-Objekte über das `wanderwegModel` ab und aktualisiert die Ansicht (in diesem Fall `ListActivity`) mit dieser Liste. Diese Methode wird verwendet, um die Benutzeroberfläche mit Daten zu füllen.
2. **String getSightseeingNameById(int id) :**
 - Ermittelt den Namen eines Sightseeing-Ortes anhand seiner ID unter Verwendung des `sightseeingModel`. Diese Methode wird verwendet, um den Namen eines bestimmten Sightseeing-Standorts für Anzeigezwecke zu erhalten.
3. **Wanderweg getWanderwegDetails(int id) :**
 - Holt Wanderweg-Objekt anhand seiner ID. Diese Methode bietet Zugriff auf einen Wanderweg aus der Datenbank über das `wanderwegModel`, um die Informationen über einen Wanderweg in der Benutzeroberfläche anzuzeigen.
4. **void onItemClick(Wanderweg item) :**
 - Definiert eine Aktion, wenn ein Wanderweg-Element angeklickt wird. In diesem Kontext navigiert sie zur `GuideActivity` mit der ID des ausgewählten Wanderwegs. Diese Methode wird für Benutzerinteraktionen mit einer Liste von Wanderwegs verwendet.
5. **List<String> getSightseeingNamesByIdList(List<Integer> ids) :**
 - Gibt eine Liste von Namen der Sightseeing-Objekte für gegebene IDs zurück. Diese Methode wird verwendet, um mehrere Sightseeing-Namen auf einmal abzurufen, um Listen von diesen Sightseeings in der `ListActivity` anzuzeigen.
6. **List<Integer> getSightseeingIdsByWanderweg(int wanderwegId) :**
 - Ruft eine Liste von IDs von Sightseeings ab, die mit einem bestimmten Wanderweg verbunden sind. Dies wird verwendet, um alle Sehenswürdigkeiten zu sammeln, die Teil eines bestimmten Wanderwegs sind.
7. **MarkOfSightseeing getMarkOfSightseeingById(int mark) :**

- Holt ein `MarkOfSightseeing`-Objekt anhand seiner ID aus der Datenbank über das `sightseeingModel`, um dann die Marker auf einer Karte anzuzeigen.
- 8. **`Sightseeing getSightseeingDetails(int id) :`**
 - Ruft `Sightseeing`-Objekt mit allen dazugehörigen Informationen, die in der `Sightseeing`-Tabelle gespeichert sind. Diese Methode wird verwendet, um detaillierte Daten über den `Sightseeing`-Ort in der Benutzeroberfläche zu präsentieren.
- 9. **`void loadMarkers() :`**
 - Lädt Markierungen (für Wanderwege) aus der Datenbank über das `markOfWanderwegModel` und fügt sie der Kartenansicht in `MapActivity` hinzu. Diese Methode ist für die Visualisierung der Wanderwege auf einer Karte verantwortlich.
- 10. **`List<Wanderweg> getWanderwegList() :`**
 - Hilfsmethoden für den test mit mocking

Interface `IComRatController`:

Dieses Interface befasst sich mit der Verwaltung von Benutzerfeedback in Form von Kommentaren und Bewertungen zu den Wanderwegen.

1. **`void submitComment(String name, String commentText, Runnable onSuccess, Runnable onFailure) :`**
 - Diese Methode ist für das Speichern des Kommentars in der Datenbank verantwortlich. Sie nimmt den Namen des Benutzers und einen Kommentartext entgegen. Wenn einer der beiden leer ist, wird die `onFailure`-Aktion ausgeführt. Andernfalls wird ein neues `Comment`-Objekt erstellt und über das `ICommentModel` in die Datenbank eingefügt. Bei erfolgreichem Einfügen wird `onSuccess` ausgeführt.
2. **`void submitRating(String name, int ratingValue, int wanderwegId, Runnable onSuccess, Runnable onFailure) :`**
 - Diese Methode ist für das Speichern von Ratings in der Datenbank verantwortlich. Sie nimmt den Namen des Benutzers, einen Bewertungswert und die ID eines Wanderwegs entgegen. Wenn der Name leer ist oder die Wanderweg-ID ungültig (-1) ist, wird die `onFailure`-Aktion ausgeführt. Andernfalls wird ein neues `Rating`-Objekt erstellt und über das `IRatingModel` in die Datenbank eingefügt. Nach dem Einfügen der Bewertung wird die Durchschnittsbewertung des entsprechenden Wanderwegs aktualisiert, und bei Erfolg wird `onSuccess` ausgeführt.

Interfaces von WanderwegeModel

Die Interfaces, die hier definiert sind, sind ein integraler Bestandteil der Datenmodellschicht in dieser Anwendung. Jedes Interface definiert spezifische Operationen, die für die Verwaltung, Abfrage und Aktualisierung von Daten in der Anwendung erforderlich sind. Es wurden 5 Interfaces definiert, damit der Code leicht zu verstehen und zu warten ist und damit jedes Interface und dann die implementierten Klassen sich nur um eine Sache kümmern.

Interface ICommentModel

Dieses Interface behandelt die Logik zum Einfügen von Kommentaren in die Datenbank.

1. **`void insertComment(Comment comment, Runnable onSuccess, Runnable onFailure) :`**

- Fügt einen `Comment`-Eintrag in die Datenbank über das `CommentDao`-Interface ein. Dies geschieht asynchron mit einem `ExecutorService`. Bei erfolgreichem Einfügen wird die `onSuccess`-Methode ausgeführt, bei einem Fehler die `onFailure`-Methode. Diese Methode ermöglicht es, Kommentare zu speichern und entsprechende Aktionen bei Erfolg oder Misserfolg durchzuführen.

Interface IRatingModel

Dieses Interface behandelt die Logik zum Einfügen von Bewertungen in die Datenbank und zur Berechnung der durchschnittlichen Bewertung.

1. **`void insertRating(Rating rating, Runnable onSuccess, Runnable onFailure) :`**

- Ähnlich wie `insertComment`, fügt diese Methode eine Bewertung (`Rating`) asynchron in die Datenbank über das `RatingDao`-Interface ein. Bei Erfolg wird `onSuccess` und bei einem Fehler `onFailure` aufgerufen. Diese Methode ermöglicht es, Ratings zu speichern und entsprechende Aktionen bei Erfolg oder Misserfolg durchzuführen.

2. **`void calculateAndUpdateAverageRating(int wanderwegId) :`**

- Berechnet den Durchschnittswert aller Bewertungen für einen bestimmten Wanderweg und aktualisiert diesen Wert in der Datenbank. Dies geschieht asynchron mit einem `ExecutorService` und über das `RatingDao`-Interface.

Interface IMarkOfWanderwegModel

Dieses Interface verwaltet die Markierungen der Wanderwege.

1. **getAllWanderwegsMarks()** :

- Diese Methode gibt eine Liste aller Markierungen (`MarkOfWanderweg`) für Wanderwege zurück. Die Daten werden direkt aus der Datenbank über `MarkOfWanderwegDao-Interface` abgefragt. Diese Methode wird verwendet, um alle vorhandenen Markierungen von Wanderwegen auf einer Karte anzuzeigen oder in einer Liste zu verarbeiten.

Interface ISightseeingModel

Dieses Interface steuert die Sightseeing-bezogenen Daten.

1. **Sightseeing getSightseeingById(int id)** :

- Diese Methode liefert ein `Sightseeing`-Objekt basierend auf einer ID. Sie verwendet die `SightseeingDao-Interface`, um die entsprechenden Daten zu abzurufen.

2. **String getSightseeingNameById(int id)** :

- Gibt den Namen einer Sehenswürdigkeit anhand ihrer ID zurück. Sie verwendet die `SightseeingDao-Interface`, um die entsprechenden Daten zu abzurufen.

3. **List<String> getSightseeingNames(List<Integer> ids)** :

- Ermittelt den Namen mehrerer Sehenswürdigkeiten, die durch eine Liste von IDs angegeben sind. Sie wird verwendet, um eine Sammlung von Namen effizient abzurufen. Diese Methode verwendet `getSightseeingNameById(int id)` - Methode, um es zu ermöglichen.

4. **MarkOfSightseeing getMarkOfSightseeingById(int markId)** :

- Liefert eine Markierung (`MarkOfSightseeing`) für eine Sehenswürdigkeit anhand ihrer ID. Dies ist wichtig für die Lokalisierung und für das Setzen von Markierungen von Sehenswürdigkeiten auf einer Karte.

5. **void createAndAddSightseeings()** :

- Erstellt eine Reihe von `Sightseeing`-Objekten und fügt sie in die Datenbank ein. Diese Methode wird verwendet, um die Datenbank mit vordefinierten Sehenswürdigkeiten zu initialisieren. Es erfolgt über `MarkOfSightseeingDao-Interface`.

6. **void addGeoPointsForSightseeings () :**
 - Erstellt geografische Punkte (`GeoPoint`) aus der osmdroid-Bibliothek für verschiedene Sehenswürdigkeiten und speichert diese Informationen in der Datenbank. Dies dient dazu, die geografische Position jeder Sehenswürdigkeit festzulegen.

Interface IWadnerwegModel

Dieses Interface befasst sich mit der Verwaltung von Wanderwegdaten.

1. **List<Wanderweg> getAllWanderwegs () :**
 - Gibt eine Liste aller Wanderwege zurück. Diese Methode greift auf die Datenbank zu über `WanderwegDao`-Interface und holt alle vorhandenen Wanderweg-Einträge.
2. **Wanderweg getWanderwegById(int id) :**
 - Liefert einen spezifischen Wanderweg basierend auf seiner ID. Diese Methode ist nützlich, um detaillierte Informationen über einen bestimmten Wanderweg zu erhalten. Es erfolgt über `WanderwegDao`-Interface.
3. **List<Integer> getSightseeingIdsByWanderwegId(int wanderwegId) :**
 - Gibt eine Liste von Sightseeing-IDs zurück, die einem spezifischen Wanderweg zugeordnet sind. Diese Methode hilft dabei, die Sightseeing-Orte zu identifizieren, die entlang eines Wanderwegs liegen. Es erfolgt über `WanderwegDao`-Interface.
4. **void createAndAddWanderwegs () :**
 - Erstellt vordefinierte Wanderwege und Sightseeings und fügt sie der Datenbank hinzu. Diese Methode wird verwendet, um die Datenbank mit vordefinierten Wanderwegen zu initialisieren. Es erfolgt über `WanderwegDao`- und `SightseeingDao`-Interface.
5. **void addGeoPointsForWanderwegs () :**
 - Fügt geografische Punkte (Geopunkte) für die definierten Wanderwege hinzu. Diese Methode ist entscheidend für die Positionierung der Markierungen von existierenden Wanderwegen auf einer Karte. Es erfolgt über `MarkOfWanderwegDao`-Interface.

Interface von MapHandler

Das Interface `IMapHandler` ist speziell für die Verwaltung und Interaktion mit einer Kartenansicht für diese Anwendung definiert-

Interface IMapHandler

1. **void setMapCenter(double latitude, double longitude, int zoomLevel) :**
 - Setzt das Zentrum der Karte auf einen bestimmten geografischen Punkt (bestimmt durch Breiten- und Längengrade) und passt das Zoomniveau an. Diese Methode wird verwendet, um die Karte auf einen spezifischen Ort zu zentrieren.
2. **void addWanderwegMarkerToMap(List<MarkOfWanderweg> locations) :**
 - Fügt der Karte Marker für eine Liste von Wanderweg-Markierungen hinzu. Jeder Marker wird mit einer Aktion versehen, die per Klick ausgeführt wird und eine neue Aktivität (GuideActivity) mit Informationen über den jeweiligen Wanderweg startet.
3. **void addSightseeingMarkerToMap(MarkOfSightseeing mark, int sightseeingId) :**
 - Platziert einen Marker für eine Sehenswürdigkeit auf der Karte. Ähnlich wie bei den Wanderweg-Markern öffnet ein Klick auf diesen Marker eine SSGuideActivity.
4. **void requestPermissionsIfNecessary(String[] permissions) :**
 - Überprüft, ob die notwendigen Berechtigungen für die Kartenfunktionen erteilt sind und fordert sie bei Bedarf an. Diese Methode ist wichtig, um sicherzustellen, dass die App ordnungsgemäß auf Kartenfunktionen zugreifen kann.

Interfaces von WanderwegeSpeicher

Die Interfaces von `WanderwegeSpeicher` definieren die Datenzugriffsschicht (Data Access Layer) der Anwendung. Sie bieten spezifische Methoden zum Interagieren mit der Datenbank für verschiedene Entitäten wie Kommentare, Markierungen, Bewertungen, Sehenswürdigkeiten und Wanderwege.

CommentDao

Dieses Interface dient zur Verwaltung von Kommentaren in der Datenbank.

1. **void insertComment(Comment comment) :**
 - Fügt einen neuen Kommentar in die Datenbank ein.
2. **getCommentById(int commentId) :**
 - Ruft einen spezifischen Kommentar basierend auf seiner ID ab.

MarkOfSightseeingDao

Dieses Interface behandelt die Markierungen von Sehenswürdigkeiten.

1. **MarkOfSightseeing getMarkOfSightseeingById(int markId) :**
 - Ruft eine spezifische Markierung für eine Sehenswürdigkeit basierend auf ihrer ID ab.
2. **void insert(MarkOfSightseeing markOfSightseeing) :**
 - Fügt eine neue Markierung für eine Sehenswürdigkeit in die Datenbank ein.

MarkOfWanderwegDao

Dieses Interface konzentriert sich auf die Markierungen von Wanderwegen.

1. **getAllWanderwegsMarks() :**
 - Ruft alle Markierungen für Wanderwege ab.
2. **insert(MarkOfWanderweg markOfWanderweg) :**
 - Fügt eine neue Markierung für einen Wanderweg in die Datenbank ein.

RatingDao

Dieses Interface befasst sich mit der Verwaltung von Bewertungen.

1. **void insertRating(Rating rating) :**
 - Fügt eine neue Bewertung in die Datenbank ein.
2. **List<Rating> getRatingsForWanderweg(int wanderwegId) :**
 - Ruft alle Bewertungen für einen bestimmten Wanderweg ab.
3. **void updateAverageRating(int wanderwegId, double averageRating) :**
 - Aktualisiert die durchschnittliche Bewertung für einen bestimmten Wanderweg.

SightseeingDao

Dieses Interface verwaltet die Daten von Sehenswürdigkeiten.

1. **List<String> getSightseeingListOfNamesByIds(List<Integer> sightseeingIds) :**
 - Ruft die Namen einer Liste von Sehenswürdigkeiten basierend auf ihren IDs ab.
2. **Sightseeing getSightseeingById(int sightseeingId) :**
 - Ruft eine spezifische Sehenswürdigkeit basierend auf ihrer ID ab.
3. **String getSightseeingNameById(int sightseeingId) :**
 - Ruft den Namen einer spezifischen Sehenswürdigkeit basierend auf ihrer ID ab.
4. **void insertAll(List<Sightseeing> sightseeingList) :**
 - Fügt eine Liste von Sehenswürdigkeiten in die Datenbank ein.
5. **List<Sightseeing> getSightseeingsByIds(List<Integer> sightseeingIds) :**

- Ruft Sehenswürdigkeiten basierend auf einer Liste von IDs ab.

WanderwegDao

Dieses Interface befasst sich mit der Verwaltung von Wanderwegen.

1. **Wanderweg getWanderwegById(int wanderwegId) :**
 - Diese Methode holt einen spezifischen Wanderweg aus der Datenbank basierend auf seiner ID. Sie wird verwendet, um detaillierte Informationen über einen bestimmten Wanderweg abzurufen.
2. **List<Wanderweg> getAllWanderwegs() :**
 - Diese Methode gibt eine Liste aller Wanderwege in der Datenbank zurück. Sie wird genutzt, um eine Übersicht über alle verfügbaren Wanderwege zu erhalten.
3. **String getSSingIdsByWanderwegId(int wanderwegId) :**
 - Ermittelt die IDs der Sehenswürdigkeiten, die einem spezifischen Wanderweg zugeordnet sind. Diese Methode ist wichtig, um die mit einem Wanderweg verbundenen Sightseeing-Punkte zu identifizieren.
4. **void insertAll(List<Wanderweg> wanderwegList) :**
 - Fügt eine Liste von Wanderwegen in die Datenbank ein. Diese Methode wird typischerweise verwendet, um neue Wanderwege zu

6. End-To-End-Tests

Use Case 1:

Name: Start von der Anwendung und Auswahl aus einer Liste

Beschreibung: Die App wird gestartet, ein Wanderweg wird aus der Liste ausgewählt, die Details werden überprüft, man kann sich auch alle einzelnen Sehenswürdigkeiten anschauen

Akteure: Benutzer, Anwendung

Precondition: Die App ist installiert und gestartet.

Post-Condition: Der Benutzer hat die Details eines Wanderwegs aus der Liste angezeigt.

Schritte:

1. Der Benutzer startet die Anwendung.
2. Der Benutzer klickt auf den Button "Aus der Liste auswählen"
3. Die Liste der verfügbaren Wanderwege wird auf dem Screen angezeigt.
4. Der Benutzer wählt einen Wanderweg aus der Liste aus.
5. Die Details des ausgewählten Wanderwegs werden angezeigt.
6. Der Benutzer kann sich alle Sehenswürdigkeiten aus diesem Wanderweg einzeln anschauen.

Test-Fälle:

- Überprüfen, ob die Anwendung startet, Homescreen angezeigt wird, man auf Button klicken kann und die Liste der Wanderwege angezeigt wird.
- Einen Wanderweg auswählen und überprüfen, ob die Details korrekt angezeigt werden.
- Einzelne Sehenswürdigkeiten auswählen und überprüfen, ob die Details korrekt angezeigt werden

Use Case 2:

Name: Start von der Anwendung und Auswahl auf der Karte

Beschreibung: Die App wird gestartet, ein Wanderweg wird auf der Karte ausgewählt, die Details werden überprüft, die Details werden überprüft, man kann sich auch alle einzelnen Sehenswürdigkeiten anschauen

Akteure: Benutzer, Anwendung

Precondition: Die App ist installiert und gestartet.

Post-Condition: Der Benutzer hat einen Wanderweg auf der Karte ausgewählt und die Details dieses Wanderwegs werden angezeigt. Man ist auch in der Lage, sich jede einzelne Sehenswürdigkeit anzuschauen.

Schritte:

1. Der Benutzer startet die Anwendung.
2. Der Benutzer klickt auf den Button "Auf der Liste auswählen"
3. Die Karte mit den gekennzeichneten Wanderwege wird auf dem Screen angezeigt.
4. Der Benutzer wählt einen Wanderweg auf der Karte aus.
5. Die Details des ausgewählten Wanderwegs werden angezeigt.
6. Der Benutzer kann sich alle Sehenswürdigkeiten aus diesem Wanderweg einzeln anschauen.

Test-Fälle:

- Überprüfen, ob die Kartenansicht beim Start der Anwendung korrekt geladen wird.
- Einen Wanderweg auf der Karte auswählen und überprüfen, ob die Details richtig angezeigt werden.

Use Case 3:

Name: Der Wanderweg wird bewertet

Beschreibung: Die App wird gestartet, ein Wanderweg wird aus einer Liste ausgewählt, Sehenswürdigkeiten werden angeschaut und dann wird man den Wanderweg bewerten.

Akteure: Benutzer, Anwendung

Precondition: Die App ist installiert und gestartet.

Post-Condition: Der Benutzer hat einen Wanderweg aus einer Liste ausgewählt und die Details dieses Wanderwegs werden angezeigt. Man kann diesen bewerten, indem man den Rating-Button tippt und auch seinen Namen und Vornamen eingibt. Die Daten sind dann korrekt gespeichert.

Schritte:

1. Der Benutzer startet die Anwendung.
2. Der Benutzer klickt auf den Button "Aus der Liste auswählen"
3. Die Liste mit den gekennzeichneten Wanderwege wird auf dem Screen angezeigt.
4. Die Details des ausgewählten Wanderwegs werden angezeigt.
5. (Der Benutzer kann sich alle Sehenswürdigkeiten aus diesem Wanderweg einzeln anschauen - muss nicht)
6. Der Benutzer tippt jetzt auf "+"-Button, da er diesen Wanderweg bewerten möchte.
7. Ein Screen mit 2 Buttons wird angezeigt
8. Der Benutzer klickt auf dem "Rate your experience"-Button
9. Der Benutzer wählt seine Bewertung aus und tippt auf diesen Button, dabei gibt er auch seinen Namen und Vornamen ein.

Test-Fälle:

- Überprüfen, ob die Sehenswürdigkeit auswählbar ist.
- Einen Kommentar hinterlassen und überprüfen, ob dieser erfolgreich gespeichert wird.

Use Case 4:

Name: Für den Wanderweg wird ein Kommentar hinterlassen.

Beschreibung: Die App wird gestartet, ein Wanderweg wird auf der Karte ausgewählt, eine Sehenswürdigkeit wird aus der Liste ausgewählt und dann wird ein Kommentar gelassen .

Akteure: Benutzer, Anwendung

Precondition: Die App ist installiert und gestartet.

Post-Condition: Der Benutzer hat einen Wanderweg auf der Karte/aus einer Liste ausgewählt und die Details dieses Wanderwegs werden angezeigt. Man schaut sich jede einzelne Sehenswürdigkeit an und kann diese kommentieren, indem man einen Kommentar schreibt und auch seinen Namen und Vornamen eingibt. Die Daten sind dann korrekt gespeichert.

Schritte:

1. Der Benutzer startet die Anwendung.
2. Der Benutzer klickt auf den Button "Aus der Liste auswählen" (ist auch über die Karte möglich die gleiche Schritten zu machen)
3. Die Liste mit den gekennzeichneten Wanderwege wird auf dem Screen angezeigt.
4. Der Benutzer wählt einen Wanderweg aus der Liste aus.
5. Die Details des ausgewählten Wanderwegs werden angezeigt.
6. (Der Benutzer kann sich alle Sehenswürdigkeiten aus diesem Wanderweg einzeln anschauen - muss nicht)
7. Der Benutzer tippt jetzt auf "+"-Button, da er für diesen Wanderweg einen Kommentar schreiben möchte.
8. Ein Screen mit 2 Buttons wird angezeigt.
9. Der Benutzer klickt auf den "Leave a comment!"-Button
10. Der Benutzer schreibt einen Kommentar und gibt auch seinen Namen und Vornamen ein.

Test-Fälle:

- Überprüfen, ob die Sehenswürdigkeit auswählbar ist.
- Eine Bewertung hinterlassen und überprüfen, ob dieser erfolgreich gespeichert wird.